

Zero-Trust DevSecOps Pipeline

Software Supply Chain Security, Keyless Signing & Policy-as-Code — Phase-by-Phase Implementation Guide

September 2026

Contents

Phase 0 — Prerequisites and tool installation	3
0.1 What you are installing and why	3
0.2 macOS — install Homebrew first	3
0.3 Windows — winget for the core, Chocolatey for the security tools	3
0.4 Verify every tool	4
0.5 Authenticate	5
Phase 1 — Architecture and repository layout	5
1.1 The concept: what Zero-Trust means for a pipeline	5
1.2 The pipeline flow	6
1.3 Tech stack	6
1.4 Repository layout	7
1.5 Who owns what	7
1.6 Set up the repository (Dev D)	7
1.7 Interview questions — Phase 1	8
Phase 2 — Security infrastructure as code (Terraform)	9
2.1 The concept: OIDC, and why it kills static secrets	9
2.2 Set the billing guardrail first	9
2.3 Create the Terraform state backend	9
2.4 The Terraform code	10
2.5 Apply	12
2.6 Publish the two variables (this unblocks Dev D)	12
2.7 Verify the trust scope	12
2.8 Interview questions — Phase 2	12
Phase 3 — Microservice and secure Dockerfile	13
3.1 The concept: Go, distroless, and non-root	13
3.2 Set the Go module path (one time, then a pull request)	13
3.3 The Dockerfile	14
3.4 Build and test	15
3.5 Prove the hardening (do not just trust it)	15
3.6 Interview questions — Phase 3	16
Phase 4 — The six-gate GitHub Actions pipeline	16
4.1 The concept: how the gates chain	16
4.2 The workflow structure	17
4.3 Rehearse every gate locally first	17
4.4 Run the pipeline	18
4.5 The step that usually costs a day	18

4.6 Interview questions — Phase 4	18
Phase 5 — Policy enforcement and gatekeeping (OPA/Rego)	19
5.1 The concept: OPA, Rego, and Conftest	19
5.2 The rules	19
5.3 Run the policy tests	21
5.4 Interview questions — Phase 5	21
Phase 6 — Vulnerability management and dashboard	21
6.1 The concept: SARIF and centralised findings	21
6.2 Enable Code Scanning	22
6.3 The summary workflow	22
6.4 Test it locally	22
6.5 Trigger the workflow manually	22
6.6 Optional — DefectDojo	22
6.7 Interview questions — Phase 6	23
Phase 7 — Security audit and bypass testing	23
7.1 The concept: adversarial validation	23
7.2 Test 1 — a vulnerable dependency	23
7.3 Test 2 — a container running as root	24
7.4 Test 3 — an unsigned image	24
7.5 Deploy-time verification	24
7.6 Evidence to capture	25
7.7 Interview questions — Phase 7	25
Phase 8 — LinkedIn showcase	25
8.1 The post	25
8.2 Posting tips	25
8.3 Make the repository public	26
8.4 One thing that costs nothing and matters a lot	26
Cost and cleanup	26
What this costs	26
Teardown (Dev A)	26
Troubleshooting	27
You are done	28

A production-grade CI/CD pipeline where no container image reaches the registry without cryptographic proof of what it is, what is inside it, and who built it.

This guide takes a team of four from empty machines to a working, self-defending pipeline on AWS, in eight phases. It is written to be beginner-friendly and gives commands for both **macOS** and **Windows (PowerShell)** wherever they differ.

How to read this guide. Commands appear in labelled blocks — **macOS** and **Windows (PowerShell)**. When a command is identical on both systems it is shown once under **Both**. Copy the block that matches your machine.

Each phase follows the same shape: **the concept in plain English**, then **the code and exact commands**, then **two or three interview questions and answers**. The phases are ordered by dependency, so work through them in sequence.

A note for Windows users. Three scripts in this project (tests/audit/*.sh) are bash scripts. Run them in **Git Bash**, which installs automatically with Git for Windows. Everything else works natively in PowerShell.

Phase 0 — Prerequisites and tool installation

Every team member does this on their own machine. Only Dev A needs working AWS credentials.

0.1 What you are installing and why

Tool	Why you need it
Git	Version control; on Windows it also provides Git Bash
Go 1.22+	Builds the microservice
Docker	Builds and runs the container image
Terraform	Provisions AWS infrastructure as code
AWS CLI	Authenticates and inspects AWS resources
GitHub CLI (gh)	Creates the repo, sets variables, watches pipeline runs
jq	Formats JSON output readably
OPA + Conftest	Runs the Policy-as-Code rules
Trivy	Scans the container image for vulnerabilities
Syft	Generates the Software Bill of Materials
Cosign	Signs the image without a private key
Hadolint	Lints the Dockerfile for security issues

0.2 macOS — install Homebrew first

Homebrew is the standard macOS package manager. If you do not have it yet:

```
# macOS
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Then install everything in two lines:

```
# macOS
brew update
brew install git go terraform awscli gh jq opa conftest trivy syft cosign hadolint
brew install --cask docker
```

Launch Docker Desktop once from Applications and wait until the whale icon stops animating. Docker will not work from the terminal until the desktop app is running.

0.3 Windows — winget for the core, Chocolatey for the security tools

Windows 10/11 ships with `winget`, but it does not carry most of the security scanners, so you will use Chocolatey for those. You need both.

Step 1 — core tools:

```
# Windows (PowerShell)
winget install --exact --id Git.Git
winget install --exact --id GoLang.Go
winget install --exact --id Hashicorp.Terraform
winget install --exact --id Amazon.AWSCLI
winget install --exact --id GitHub.cli
winget install --exact --id jqlang.jq
winget install --exact --id Docker.DockerDesktop
```

Step 2 — install Chocolatey (PowerShell as Administrator):

```
# Windows (PowerShell, as Administrator)
Set-ExecutionPolicy Bypass -Scope Process -Force
[System.Net.ServicePointManager]::SecurityProtocol = 3072
iex ((New-Object
↪ System.Net.WebClient).DownloadString('https://community.chocolatey.org/install.ps1'))
```

Step 3 — security tools (still as Administrator):

```
# Windows (PowerShell, as Administrator)
choco install opa trivy cosign hadolint make -y
```

Step 4 — Conftest and Syft are not in Chocolatey. Create a tools folder, add it to PATH, and drop the Windows binaries in:

```
# Windows (PowerShell)
New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\bin"
[Environment]::SetEnvironmentVariable("Path", $env:Path + ";$env:USERPROFILE\bin", "User")

$ver = "0.56.0"
Invoke-WebRequest -Uri "https://github.com/open-policy-
↪ agent/conftest/releases/download/v$ver/conftest_${ver}_Windows_x86_64.zip" -OutFile
↪ "$env:TEMP\conftest.zip"
Expand-Archive "$env:TEMP\conftest.zip" -DestinationPath "$env:USERPROFILE\bin" -Force
```

For Syft, download syft_<version>_windows_amd64.zip from <https://github.com/anchore/syft/releases> and extract syft.exe into the same bin folder.

Close and re-open PowerShell so the new tools are on your PATH, then launch Docker Desktop and wait for “Engine running”.

Windows tip. If PowerShell refuses to run a script with “*running scripts is disabled on this system*”, run Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass and try again. That setting applies only to the current window.

0.4 Verify every tool

```
# Both
git --version
go version
docker --version
terraform version
aws --version
gh --version
jq --version
opa version
conftest --version
trivy --version
syft version
cosign version
hadolint --version
```

A faster all-in-one check:

```
# macOS
for t in git go docker terraform aws gh jq opa conftest trivy syft cosign hadolint; do
  printf "%-12s " "$t"; command -v "$t" >/dev/null 2>&1 && echo "ok" || echo "MISSING"
done
```

```
# Windows (PowerShell)
"git","go","docker","terraform","aws","gh","jq","opa","conftest","trivy","syft","cosign","hadolint" |
ForEach-Object {
    $found = Get-Command $_ -ErrorAction SilentlyContinue
    "{0,-12} {1}" -f $_, $(if ($found) {"ok"} else {"MISSING"})
}
```

Anything reporting MISSING must be fixed before Phase 1.

0.5 Authenticate

Everyone signs in to GitHub:

```
# Both
gh auth login          # choose HTTPS, authenticate via browser
gh auth status
```

Dev A only — sign in to AWS:

```
# Both
aws configure          # or: aws sso login --profile devsecops
aws sts get-caller-identity
```

That last command must print your account ID. If it errors, nothing in Phase 2 will work.

Python note. The reporting script in Phase 6 needs Python 3.10+. On macOS the interpreter is usually `python3`; on Windows it is usually `python`. Use whichever prints a 3.10+ version and substitute it wherever this guide says `python3`.

Phase 1 — Architecture and repository layout

Owner: everyone reads this. Dev D sets up the repository.

1.1 The concept: what Zero-Trust means for a pipeline

In a traditional pipeline, trust is *implicit*. Once code is inside CI, everyone assumes it is fine; artifacts flow to the registry, and whatever lands in the registry gets deployed. That assumption is exactly how software supply chain attacks succeed — an attacker slips malicious code into a dependency, or swaps an image after it was approved, and the pipeline waves it through.

Zero-Trust flips the default to “**never trust, always verify**” at every hop. Four ideas drive this whole project:

No static secrets. We never store long-lived cloud keys or registry passwords in GitHub. Instead GitHub Actions proves its identity to AWS using short-lived **OIDC tokens** — think of a temporary ID badge issued per run that expires within the hour. This is why Phase 2 provisions OIDC roles instead of access keys.

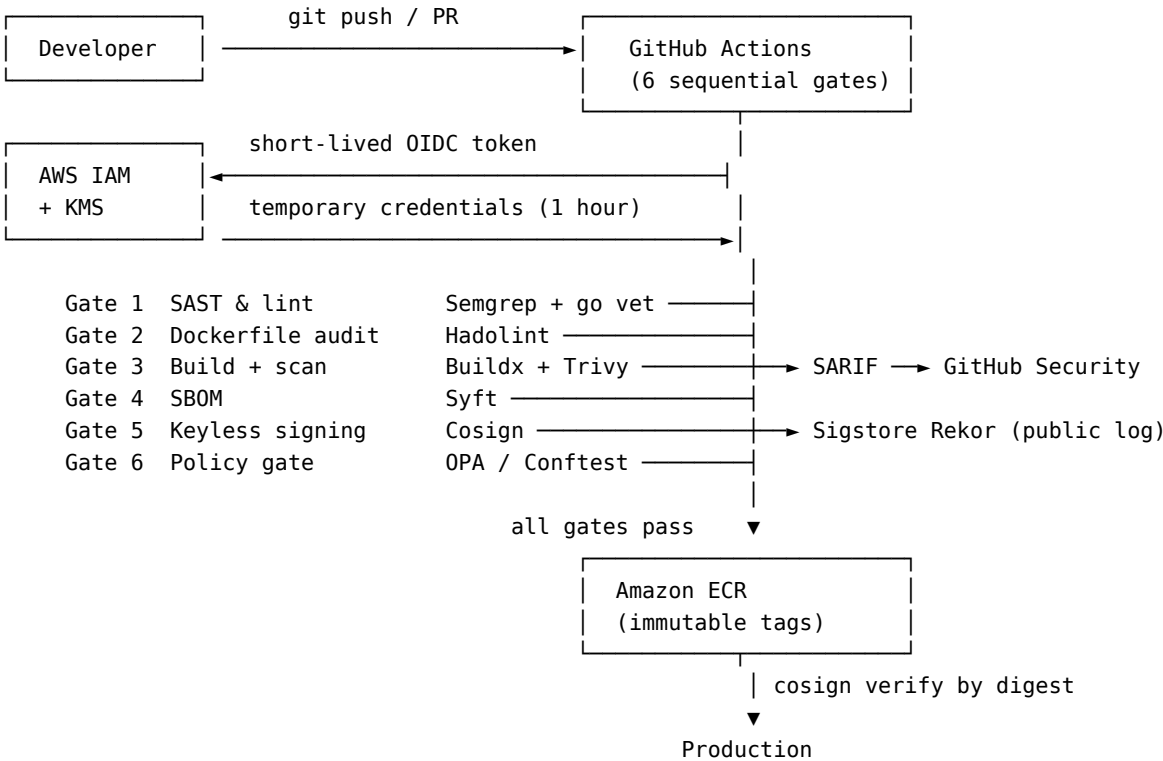
Every artifact is verifiable. A container image is not trusted because it came from CI. It is trusted because it carries cryptographic proof: a **signature** (Cosign says “we built this”), an **SBOM** (Syft says “here is every ingredient inside”), and **attestations** (signed statements binding the SBOM to the exact image digest). Verification happens by digest (`sha256:...`), never by mutable tag.

Policy is code, not tribal knowledge. Rules such as “no image may run as root” live in version-controlled `.rego` files and are enforced by a machine, not a reviewer’s memory. A build that violates policy fails.

Shift left, then gate. Cheap checks (linting, static analysis) run first and fail fast; expensive checks (build, scan, sign) run only if the cheap ones pass. Each stage is a **gate** that must be green before the next begins.

Keep this phrase for interviews: *we do not trust the pipeline; we trust cryptographic evidence attached to the artifact.*

1.2 The pipeline flow



Mermaid source for the README (renders natively on GitHub) is in docs/architecture/pipeline-flow.mermaid.

The important structural facts: the six gates are strictly sequential; the only path to the registry passes through the policy gate; and the only path to production passes through digest-based verification.

1.3 Tech stack

Layer	Technology
Cloud and registry	AWS ECR (immutable tags), AWS KMS, AWS IAM OIDC
Infrastructure as Code	Terraform (aws provider)
Application	Go 1.22, distroless base image, non-root UID 65532
Static analysis (SAST)	Semgrep, go vet
Dockerfile linting	Hadolint
Vulnerability scanning (SCA)	Trivy
SBOM generation	Syft (SPDX + CycloneDX)
Artifact signing	Cosign keyless via Sigstore (Fulcio + Rekor)
Policy-as-Code	Open Policy Agent (OPA) / Conftest, written in Rego
CI/CD	GitHub Actions
Findings dashboard	GitHub Security tab (SARIF)

1.4 Repository layout

A **monorepo** is the right choice for a four-person portfolio project: one clone, one pull request shows the whole story, and reviewers see app, infrastructure, policy and pipeline together.

```
zt-devsecops-pipeline/
├── app/                                # Phase 3 – Go microservice
│   ├── main.go                        # HTTP server, graceful shutdown
│   ├── go.mod
│   ├── main_test.go
│   └── internal/handlers/
│       ├── health.go                 # /healthz and / endpoints
│       └── metrics.go                # /metrics + observability middleware
├── build/                             # Phase 3
│   ├── Dockerfile                    # Multi-stage, distroless, USER 65532
│   └── .hadolint.yaml
├── infra/                             # Phase 2 – Terraform (AWS)
│   ├── versions.tf                   # Providers + S3 state backend
│   ├── variables.tf
│   ├── kms.tf                        # Customer-managed KMS key
│   ├── oidc.tf                       # GitHub OIDC provider + scoped IAM role
│   ├── registry.tf                  # ECR with immutable tags
│   └── outputs.tf
├── policy/                            # Phase 5 – Policy-as-Code
│   ├── container.rego
│   ├── attestation.rego
│   └── tests/                         # 11 Rego unit tests + fixtures
├── security/
│   ├── trivy/                        # Severity thresholds, ignore file
│   └── cosign/verify.sh              # Deploy-time verification
├── scripts/                           # Phase 6 – findings automation (Python)
├── tests/audit/                       # Phase 7 – three negative tests (bash)
├── .github/workflows/                 # Phase 4 – the 6-gate pipeline
├── Makefile
└── docs/                             # This guide, TEAM.md, DAY0-SETUP.md
```

1.5 Who owns what

Dev	Role	Owns	Phases
A	Cloud and identity	infra/	2
B	App and container	app/, build/	3
C	Policy and findings	policy/, security/, scripts/	5, 6
D	Pipeline and audit	.github/, tests/audit/	4, 7

Dev A is on the critical path — Dev D cannot run the cloud-dependent gates until Dev A publishes two repository variables. A, B and C all start on day one in parallel; Dev D spends day one rehearsing each gate locally.

Dev A owns the AWS account and the bill (about \$1–3 per month). Dev D is the repository admin.

1.6 Set up the repository (Dev D)

Nobody forks. The AWS trust policy is scoped to exactly one repository, so forks cannot authenticate.

```
# Both
cd zt-devsecops-pipeline

git branch -m master main          # rename if git created "master"
git branch --show-current          # must print: main

gh repo create <ORG>/zt-devsecops-pipeline \
  --private --source=. --remote=origin --push
```

The branch **must** be main. Both the workflow trigger and the AWS OIDC trust policy target refs/heads/main; if the branch is master, nothing runs and AWS rejects the token.

Protect the branch so every change is reviewed:

```
# Both
gh api -X PUT repos/<ORG>/zt-devsecops-pipeline/branches/main/protection \
  -H "Accept: application/vnd.github+json" \
  -f "required_pull_request_reviews[required_approving_review_count]=1" \
  -F "enforce_admins=false" -F "required_status_checks=null" -F "restrictions=null"
```

Invite the other three with Write access:

```
# Both
for u in <devA> <devB> <devC>; do
  gh api -X PUT repos/<ORG>/zt-devsecops-pipeline/collaborators/$u -f permission=push
done
```

Everyone else clones:

```
# macOS
cd ~/projects
git clone https://github.com/<ORG>/zt-devsecops-pipeline.git
cd zt-devsecops-pipeline
```

```
# Windows (PowerShell)
cd $env:USERPROFILE\projects
git clone https://github.com/<ORG>/zt-devsecops-pipeline.git
cd zt-devsecops-pipeline
```

1.7 Interview questions — Phase 1

Q1. What does Zero-Trust actually change in a CI/CD pipeline? It removes implicit trust between stages. A normal pipeline assumes anything inside CI or in the registry is safe. Zero-Trust requires verifiable evidence at each hop: identity is proven per run via short-lived OIDC tokens instead of stored secrets, artifacts must carry signatures, SBOMs and attestations, and deployment verifies the image by immutable digest against a policy — not by trusting a tag or the fact that CI produced it.

Q2. Why verify a container by its sha256 digest instead of a tag like :latest or :v1.2? Tags are mutable — someone can repoint a tag to a different, malicious image after it was scanned and signed. That is a tag-mutation attack. The digest is a content hash: it uniquely and immutably identifies the exact bytes that were scanned and signed. Enabling immutable tags in the registry plus verifying by digest closes that gap, so what you verified is provably what you deploy.

Q3. Monorepo or polyrepo here, and why? Monorepo for this project: application, infrastructure, policy and pipeline evolve together, a single pull request tells the whole security story, and one CI configuration governs everything. Polyrepo makes sense at organisational scale where independent teams need separate release cadences and access control; the cost is duplicated pipeline

and policy tooling, and harder cross-cutting changes.

Phase 2 — Security infrastructure as code (Terraform)

Owner: Dev A. This phase blocks Dev D — finish it first.

2.1 The concept: OIDC, and why it kills static secrets

The old, dangerous way: create an IAM user, generate an `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`, and paste them into GitHub Secrets. Those keys are long-lived, rarely rotated, and if leaked — through a logged environment variable or a compromised action — an attacker has standing access to your account. This is the single most common cloud breach vector.

The Zero-Trust way is **OIDC federation**. GitHub Actions has its own identity provider at `token.actions.githubusercontent.com`. On every run, GitHub mints a short-lived signed **JWT** describing exactly which repository, branch and workflow is running. We tell AWS: *trust tokens from GitHub's OIDC provider, but only if the token's sub claim says it came from `repo:my-org/zt-devsecops-pipeline` on this branch*. AWS then returns temporary credentials that expire within the hour. Nothing is stored, so nothing can leak. If someone forks your repository, their sub claim differs and AWS refuses them.

This phase provisions three things:

A KMS key — a customer-managed, audit-logged encryption key used to encrypt the registry. Note that image *signing* is keyless via Cosign, so KMS here demonstrates enterprise envelope-encryption and IAM patterns rather than holding a signing key.

The OIDC provider and IAM role — with a *trust policy* (who may assume it, scoped to your repository and branch) and a *permission policy* (what they may do — push to one ECR repository and nothing else).

The ECR repository — with **immutable tags**, so a tag can never be repointed after push, plus scan-on-push and a lifecycle rule that expires untagged images.

2.2 Set the billing guardrail first

Expected cost is \$1–3 per month, but set the alarm before you create anything:

```
# macOS
ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
aws budgets create-budget --account-id "$ACCOUNT" --budget '{
  "BudgetName": "zt-devsecops",
  "BudgetLimit": {"Amount": "5", "Unit": "USD"},
  "TimeUnit": "MONTHLY", "BudgetType": "COST"
}'
```

```
# Windows (PowerShell)
$ACCOUNT = aws sts get-caller-identity --query Account --output text
$budget = '{"BudgetName": "zt-devsecops", "BudgetLimit": {"Amount": "5", "Unit": "USD"}, "TimeUnit": "
↪ "MONTHLY", "BudgetType": "COST"}'
aws budgets create-budget --account-id $ACCOUNT --budget $budget
```

2.3 Create the Terraform state backend

Terraform stores its state in S3 with a DynamoDB lock table. This must be created by hand — Terraform cannot store the state that describes the bucket holding its own state.

```
# macOS
export AWS_REGION=us-east-1
export TF_BUCKET="zt-devsecops-tfstate-$(aws sts get-caller-identity --query Account --output text)"

aws s3api create-bucket --bucket "$TF_BUCKET" --region "$AWS_REGION"
aws s3api put-bucket-versioning --bucket "$TF_BUCKET" \
  --versioning-configuration Status=Enabled
aws s3api put-bucket-encryption --bucket "$TF_BUCKET" \
  --server-side-encryption-configuration \
  '{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"aws:kms"}}]}'
aws s3api put-public-access-block --bucket "$TF_BUCKET" \
  --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true

aws dynamodb create-table --table-name zt-devsecops-tflock \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST --region "$AWS_REGION"

echo "TF_BUCKET=$TF_BUCKET"

# Windows (PowerShell)
$env:AWS_REGION = "us-east-1"
$acct = aws sts get-caller-identity --query Account --output text
$env:TF_BUCKET = "zt-devsecops-tfstate-$acct"

aws s3api create-bucket --bucket $env:TF_BUCKET --region $env:AWS_REGION
aws s3api put-bucket-versioning --bucket $env:TF_BUCKET `
  --versioning-configuration Status=Enabled
aws s3api put-bucket-encryption --bucket $env:TF_BUCKET `
  --server-side-encryption-configuration
↪ '{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"aws:kms"}}]}'
aws s3api put-public-access-block --bucket $env:TF_BUCKET `
  --public-access-block-configuration
↪ BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true

aws dynamodb create-table --table-name zt-devsecops-tflock `
  --attribute-definitions AttributeName=LockID,AttributeType=S `
  --key-schema AttributeName=LockID,KeyType=HASH `
  --billing-mode PAY_PER_REQUEST --region $env:AWS_REGION

"TF_BUCKET=$env:TF_BUCKET"
```

Windows JSON quoting. PowerShell mangles inline JSON; the backslash-escaped quotes above are deliberate. If a command still fails, write the JSON to a file and pass `--cli-input-json file://name.json`.

2.4 The Terraform code

The files are already in `infra/`. The three that matter most:

infra/oidc.tf — the trust policy. The sub condition is the single most security-critical line in this repository:

```
data "tls_certificate" "github" {
  url = "https://token.actions.githubusercontent.com/.well-known/openid-configuration"
}
```

```

resource "aws_iam_openid_connect_provider" "github" {
  url            = "https://token.actions.githubusercontent.com"
  client_id_list = ["sts.amazonaws.com"]
  thumbprint_list = [data.tls_certificate.github.certificates[0].sha1_fingerprint]
}

data "aws_iam_policy_document" "github_trust" {
  statement {
    effect = "Allow"
    actions = ["sts:AssumeRoleWithWebIdentity"]

    principals {
      type       = "Federated"
      identifiers = [aws_iam_openid_connect_provider.github.arn]
    }

    condition {
      test      = "StringEquals"
      variable = "token.actions.githubusercontent.com:aud"
      values    = ["sts.amazonaws.com"]
    }

    # This is the line that stops forks and other repositories.
    condition {
      test      = "StringLike"
      variable = "token.actions.githubusercontent.com:sub"
      values    = [for ref in var.allowed_branches :
        "repo:${var.github_org}/${var.github_repo}:ref:${ref}"]
    }
  }
}

```

infra/registry.tf — immutability is enforced here:

```

resource "aws_ecr_repository" "app" {
  name                = var.ecr_repo_name
  image_tag_mutability = "IMMUTABLE" # a tag can never be repointed
  force_delete        = false

  image_scanning_configuration { scan_on_push = true }

  encryption_configuration {
    encryption_type = "KMS"
    kms_key         = aws_kms_key.devsecops.arn
  }
}

```

infra/kms.tf — note the deletion window:

```

resource "aws_kms_key" "devsecops" {
  description          = "Zero-Trust DevSecOps platform KMS key"
  deletion_window_in_days = var.kms_deletion_window_days # default 7, not 30
  enable_key_rotation  = true
}

```

A KMS key pending deletion is **still billed** for the whole window, which is why the default is 7 days rather than the 30-day maximum.

2.5 Apply

```
# macOS
cd infra
cp terraform.tfvars.example terraform.tfvars
# edit terraform.tfvars: set github_org to your real org
```

```
# Windows (PowerShell)
cd infra
Copy-Item terraform.tfvars.example terraform.tfvars
notepad terraform.tfvars
```

```
# macOS
terraform init -backend-config="bucket=$TF_BUCKET" -backend-config="region=$AWS_REGION"
```

```
# Windows (PowerShell)
terraform init -backend-config="bucket=$env:TF_BUCKET" -backend-config="region=$env:AWS_REGION"
```

```
# Both
terraform fmt -check
terraform validate
terraform plan -out=tfplan      # READ THIS before applying
terraform apply tfplan
```

2.6 Publish the two variables (this unblocks Dev D)

```
# Both
terraform output -raw github_actions_role_arn
terraform output -raw ecr_repository_url

gh variable set AWS_ROLE_ARN --body "${terraform output -raw github_actions_role_arn}"
gh variable set ECR_REPOSITORY --body "${terraform output -raw ecr_repository_url}"
gh variable list
```

These are **Variables**, not Secrets — they are not sensitive.

2.7 Verify the trust scope

Do this before moving on. It is the most important check in the project:

```
# Both
aws iam get-role --role-name zt-devsecops-github-actions \
  --query 'Role.AssumeRolePolicyDocument' | jq .
```

The sub condition must read `repo:<ORG>/zt-devsecops-pipeline:ref:refs/heads/main`. If it is broader — a wildcard, or scoped only to the organisation — **stop and fix it**. Without that condition any GitHub repository in the world could assume your role.

Dev A: announce to the team that the variables are published. Dev D starts now.

2.8 Interview questions — Phase 2

Q1. Walk me through how GitHub Actions authenticates to AWS with zero stored secrets. GitHub Actions requests an OIDC JWT from `token.actions.githubusercontent.com`; the token carries claims including `sub` (repository and ref) and `aud`. The workflow calls `sts:AssumeRoleWithWebIdentity` presenting that token. AWS validates the signature against the registered OIDC provider and

checks the role's trust conditions — `aud` equals `sts.amazonaws.com`, and `sub` matches our repository and branch. On success STS returns temporary credentials valid for up to one hour. No access keys exist anywhere, so there is nothing to leak or rotate.

Q2. Your trust policy has a `StringLike` condition on the `sub` claim. Why is that line security-critical? Without it, any GitHub repository's OIDC token with the right audience could assume the role — including a fork of your project or an attacker's repository. The `sub` condition binds assumption to your specific repository and branch, enforcing least privilege on identity. A common mistake is scoping only to the organisation or using a wildcard, which effectively makes the role assumable by unintended workflows.

Q3. Why enable immutable tags on ECR, and how does that complement Cosign signing? Immutable tags prevent a pushed tag from ever being repointed to different image bytes, defeating tag-mutation attacks where a scanned and signed `:v1` is later swapped for a malicious image. It complements Cosign because signing binds trust to the content digest: immutability guarantees the tag keeps pointing at the exact digest you signed and verified. They are layered controls — registry-level integrity plus cryptographic provenance.

Phase 3 — Microservice and secure Dockerfile

Owner: Dev B. No cloud access needed — start on day one in parallel with Phase 2.

3.1 The concept: Go, distroless, and non-root

Why Go. It compiles to a single static binary with no interpreter and no runtime dependencies. The final image can therefore contain *just your binary* — no shell, no package manager, nothing for an attacker to pivot with. Fewer files means fewer CVEs for Trivy to find.

Multi-stage builds. A Dockerfile can have several `FROM` stages. The **builder** stage has the full Go toolchain (hundreds of megabytes, many packages). The **final** stage copies out only the compiled binary and discards everything else. Your shipped image never contains the compiler or its vulnerabilities.

Distroless. Google's `gcr.io/distroless/static` has no shell, no `apt`, no `busybox` — essentially just CA certificates, timezone data and `/etc/passwd`. Compare the attack surface: a `ubuntu` base has 100+ packages; `alpine` has a shell and `apk`; `distroless-static` has almost nothing. If Trivy reports zero OS-package CVEs, this is usually why.

Non-root user 65532. By default containers run as root (UID 0). If an attacker escapes your application, root inside the container plus a kernel bug can mean root on the host. Running as an unprivileged UID (`distroless` ships `nonroot = 65532`) means an escape lands as a powerless user. This is also the exact rule the Phase 5 policy enforces — the Dockerfile and the policy are two sides of the same control.

The `/healthz` and `/metrics` contract. `/healthz` is a liveness probe, so an orchestrator can restart a dead process. `/metrics` exposes Prometheus counters so the service is observable. Both are table stakes for “production-grade”.

3.2 Set the Go module path (one time, then a pull request)

The code ships with a placeholder organisation name. Fix it once for everyone — and use it as your practice run at the review workflow.

```
# macOS
git checkout -b chore/set-module-path
grep -rl 'your-github-org' app/ | xargs sed -i '' 's|your-github-org|<ORG>|g'
```

```
# Windows (PowerShell)
git checkout -b chore/set-module-path
Get-ChildItem -Path app -Recurse -File |
  ForEach-Object {
    (Get-Content $_.FullName) -replace 'your-github-org', '<ORG>' | Set-Content $_.FullName
  }
```

```
# Both
cd app
go mod tidy          # creates go.sum -- the Docker build fails without it
go test ./...
cd ..

git add -A
git commit -m "chore: set Go module path to the org"
git push -u origin chore/set-module-path
gh pr create --fill
```

go.sum must be committed. It pins the exact cryptographic hash of every dependency, which is itself part of the supply chain you are securing — and the Dockerfile refuses to build without it.

3.3 The Dockerfile

```
# syntax=docker/dockerfile:1.7

##### Stage 1: build #####
FROM golang:1.22-bookworm AS builder
WORKDIR /src

# Cache dependencies separately from source for faster rebuilds.
COPY app/go.mod app/go.sum ./
RUN go mod download && go mod verify

COPY app/ ./

# Static, stripped binary. CGO disabled => no libc dependency, so it runs on
# distroless/static. -trimpath removes local filesystem paths.
RUN CGO_ENABLED=0 GOOS=${TARGETOS:-linux} GOARCH=${TARGETARCH:-amd64} \
  go build -trimpath -ldflags="-s -w -buildid=" -o /out/app .

##### Stage 2: runtime #####
# distroless/static: no shell, no package manager, no libc.
# The :nonroot tag runs as UID 65532 by default.
FROM gcr.io/distroless/static-debian12:nonroot

# Redundant but explicit, so the intent is auditable and readable by OPA/Trivy.
USER 65532:65532

WORKDIR /app
COPY --from=builder --chown=65532:65532 /out/app /app/app

EXPOSE 8080

# Distroless has no shell, so ENTRYPOINT must be exec-form.
```

```
ENTRYPOINT ["/app/app"]
```

There is deliberately **no HEALTHCHECK** — distroless has no shell to run one. Liveness is done by the orchestrator over HTTP, which is the production-correct pattern anyway.

3.4 Build and test

```
# Both
cd app
go vet ./...
go test ./... -race -count=1
cd ..

docker build -f build/Dockerfile -t zt-go-microservice:dev .
```

Expect three tests to pass. Now run it and check the endpoints:

```
# macOS
docker run --rm -d -p 8080:8080 --name ztsvc zt-go-microservice:dev
curl -s localhost:8080/healthz | jq .
curl -s localhost:8080/ | jq .
curl -s localhost:8080/metrics | grep http_requests_total | head
docker rm -f ztsvc
```

```
# Windows (PowerShell)
docker run --rm -d -p 8080:8080 --name ztsvc zt-go-microservice:dev
Invoke-RestMethod http://localhost:8080/healthz
Invoke-RestMethod http://localhost:8080/
(Invoke-WebRequest http://localhost:8080/metrics).Content -split "`n" | Select-String
↪ http_requests_total | Select-Object -First 5
docker rm -f ztsvc
```

Windows tip. `curl` in PowerShell is an alias for `Invoke-WebRequest`, which takes different arguments than real `curl`. Use `Invoke-RestMethod` as shown.

3.5 Prove the hardening (do not just trust it)

Check 1 — runs as 65532, not root:

```
# macOS
docker inspect zt-go-microservice:dev --format 'User={{.Config.User}}'
```

```
# Windows (PowerShell)
docker inspect zt-go-microservice:dev --format "User={{.Config.User}}"
```

Expected on both: `User=65532:65532`

Check 2 — there is no shell. This command is *supposed* to fail:

```
# Both
docker run --rm --entrypoint /bin/sh zt-go-microservice:dev -c "echo hi"
```

An error such as “`exec: /bin/sh: not found`” is the correct, passing result.

Check 3 — the Dockerfile passes its lint gate:

```
# macOS
docker run --rm -i hadolint/hadolint hadolint --config build/.hadolint.yaml - < build/Dockerfile
```

```
# Windows (PowerShell)
Get-Content build/Dockerfile | docker run --rm -i hadolint/hadolint hadolint --config
↪ build/.hadolint.yaml -
```

Silence means success.

3.6 Interview questions — Phase 3

Q1. Explain multi-stage builds and how they reduce your image’s CVE count. A multi-stage Dockerfile uses separate FROM stages: the builder carries the full toolchain and compiles the binary, while the final stage copies out only that binary and starts from a minimal base. Scanners such as Trivy inspect only the final image, so none of the builder’s packages — and none of their CVEs — ship to production. You get a small image whose vulnerability surface is essentially just your own code.

Q2. Why distroless and non-root UID 65532 specifically, and how do they defend in depth? Distroless removes the shell and package manager, so an attacker who achieves code execution has no `sh`, `curl` or `apt` to escalate or pivot with — it dramatically shrinks post-exploitation options. Running as UID 65532 rather than root means a container escape lands as a powerless user, not host root, blunting kernel-exploit and misconfiguration attacks. They are independent layers: minimal contents *and* least privilege, so a single failure does not hand over the host. It is also enforced downstream — our OPA policy rejects any image whose configured user is root.

Q3. Your image has no shell, so how do health checks and debugging work in production? Liveness and readiness are done by the orchestrator making HTTP requests to `/healthz`, not by a `HEALTHCHECK` shell command — which is the recommended pattern regardless of base image. For debugging you avoid `docker exec`; instead you rely on structured JSON logs to stdout, Prometheus metrics from `/metrics`, and when you truly need a shell, an ephemeral debug container (`kubectl debug`) attached to the pod’s namespaces — without adding a shell to the production image.

Phase 4 — The six-gate GitHub Actions pipeline

Owner: Dev D. Partly blocked on Phase 2 — rehearse locally on day one.

4.1 The concept: how the gates chain

Fail fast, fail cheap. The gates are ordered by cost. `Semgrep` and `Hadolint` take seconds and need no build, so they run first — there is no point building an image if the source contains a hardcoded secret. Build and scan are expensive, so they run only after linting is green. Signing and policy run last because they act on the finished artifact.

needs: creates the chain. In GitHub Actions, `needs: [job-a]` means “do not start until job-a succeeded”. Wiring `sast` → `hadolint` → `build-scan` → `sbom` → `sign` → `policy` means a red gate short-circuits everything after it. That is what makes it a gate rather than a checklist.

Build once, reference by digest forever. Gate 3 builds and pushes the image, then outputs the immutable `sha256:... digest`. Every later gate operates on that exact digest — never a tag. This guarantees the thing you signed is the thing you scanned.

Keyless signing. Traditional signing needs a private key you must store and protect. `Cosign` keyless uses the same OIDC identity from Phase 2: it asks Sigstore’s **Fulcio** certificate authority for a short-lived certificate bound to the workflow’s identity, signs with an ephemeral key, records the signature in the public **Rekor** transparency log, and discards the key. No private key exists to steal.

Signatures versus attestations. A *signature* says “we vouch for this image”. An *attestation* is a signed, structured statement *about* the image — for example “here is its SBOM”. Both are pushed alongside the image and are what OPA checks in Gate 6.

Least-privilege tokens. Each job declares exactly the GitHub token scopes it needs; id-token: write to mint OIDC tokens, security-events: write only where SARIF is uploaded.

4.2 The workflow structure

The complete file is at `.github/workflows/devsecops-pipeline.yaml`. Its shape:

```
name: Zero-Trust DevSecOps Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read          # least privilege by default

env:
  AWS_REGION: us-east-1
  AWS_ROLE_ARN: ${vars.AWS_ROLE_ARN}      # published by Dev A in Phase 2
  ECR_REPOSITORY: ${vars.ECR_REPOSITORY}
  COSIGN_VERSION: v2.4.1

jobs:
  sast:          # Gate 1 – Semgrep + go vet + unit tests
  hadolint:      # Gate 2 – needs: [sast]
  build-scan:    # Gate 3 – needs: [hadolint], if: github.event_name == 'push'
  sbom:          # Gate 4 – needs: [build-scan]
  sign:          # Gate 5 – needs: [build-scan, sbom]
  policy-gate:   # Gate 6 – needs: [build-scan, sign]
```

Two design points worth understanding:

Why the four AWS jobs carry `if: github.event_name == 'push'`. The OIDC trust is scoped to `refs/heads/main`. On a pull request the token's sub claim is different, so AWS would reject it. Gating those jobs keeps pull-request runs clean — gates 1 and 2 still run and still block a bad merge — while signing happens on merge to `main`.

The critical Gate 5 step:

```
- name: Keyless sign the image (by digest)
  env:
    COSIGN_YES: "true"          # non-interactive; use the ambient OIDC token
    IMG: ${env.ECR_REPOSITORY}:${needs.build-scan.outputs.image_digest}
  run: cosign sign "$IMG"

- name: Attach SBOM attestation (CycloneDX predicate)
  env:
    COSIGN_YES: "true"
    IMG: ${env.ECR_REPOSITORY}:${needs.build-scan.outputs.image_digest}
  run: cosign attest --predicate sbom.cyclonedx.json --type cyclonedx "$IMG"
```

4.3 Rehearse every gate locally first

Do this on day one while waiting for Dev A. When a gate later fails in CI you will know whether the problem is the tool or the wiring.

```

# Gate 1 – SAST
# Both
pip install semgrep # macOS may need pip3
semgrep --config p/default --config p/golang --config p/secrets app/

# Windows (PowerShell) – if the pip install is troublesome, use Docker
docker run --rm -v "${PWD}:/src" semgrep/semgrep semgrep --config p/default --config p/golang /src/app

# Gate 2 – Dockerfile audit (Both)
docker run --rm -i hadolint/hadolint hadolint --config build/.hadolint.yaml - < build/Dockerfile

# Gate 3 – build + scan (Both)
docker build -f build/Dockerfile -t zt-local:test .
trivy image --severity CRITICAL,HIGH --ignore-unfixed --exit-code 1 zt-local:test

# Gate 4 – SBOM (Both)
syft zt-local:test -o spdx-json=sbom.spdx.json
syft zt-local:test -o cyclonedx-json=sbom.cyclonedx.json

# Gate 5 – read the signing docs before touching a real registry (Both)
cosign version
cosign sign --help

```

4.4 Run the pipeline

With Dev A's variables published and Dev B's `go.sum` merged:

```

# Both
git checkout main
git pull
git push
gh run watch

```

Debug a red gate:

```

# Both
gh run view --log-failed

```

4.5 The step that usually costs a day

Gate 6 verifies the signature against an exact expected identity:

`https://github.com/<ORG>/zt-devsecops-pipeline/.github/workflows/devsecops-pipeline.yaml@refs/heads/main`

It must match your real organisation, real workflow filename and real branch — character for character. If verification fails with “none of the expected identities matched”, print what was actually recorded and compare:

```

# Both
cosign verify <ECR_URI>@sha256:<digest> \
  --certificate-identity-regexp ".*" \
  --certificate-oidc-issuer "https://token.actions.githubusercontent.com"

```

4.6 Interview questions — Phase 4

Q1. How does `cosign sign` work with no private key, and where does the trust come from?
Keyless Cosign uses the workflow's OIDC identity. Cosign obtains an OIDC token, sends it to Sig-

store's Fulcio CA, and Fulcio issues a short-lived X.509 certificate binding an ephemeral public key to that identity. Cosign signs the image digest with the ephemeral private key, publishes the signature and certificate to the registry, records an entry in the Rekor transparency log, then discards the key. Trust at verification time comes from three things: the certificate was issued by Fulcio to the expected identity and issuer, the signature matches the digest, and Rekor proves it was logged at signing time. There is no long-lived key to protect.

Q2. Why does every gate after the build reference the image by digest instead of the tag it just pushed? The digest is the content hash — immutable, and uniquely identifying the exact bytes that were built. Scanning, SBOM generation, signing and verifying all against the same digest guarantees consistency: you sign what you scanned and verify what you signed. Referencing a tag would open a window where the tag could point at different content between steps. Immutable tags plus digest pinning close that gap completely.

Q3. What is the difference between a signature and an attestation, and why do you need both? A signature is a cryptographic vouch for the image digest — “this identity endorses this artifact”. An attestation is a signed statement *about* the artifact carrying a structured predicate, such as a CycloneDX SBOM or SLSA provenance. You need both because the signature establishes authenticity while attestations carry the verifiable metadata that the admission policy actually reasons over. Together they let a verifier answer not just “is this from us?” but “is this from us *and* does it meet policy?”

Phase 5 — Policy enforcement and gatekeeping (OPA/Rego)

Owner: Dev C. No cloud access needed.

5.1 The concept: OPA, Rego, and Conftest

OPA (Open Policy Agent) is a general-purpose policy engine. You give it *input* (JSON facts, here describing an image) and *policy* (rules written in Rego), and it answers whether the input satisfies the rules.

Rego is declarative. You do not write if/else imperatively; you write rules that produce denials. The pattern here is a set called deny that collects a message for every violation. If deny is empty, the input is compliant. If it has entries, the build is blocked and each message explains why.

Conftest is a thin CLI that runs OPA policies against configuration files. It exits non-zero if any deny fires, which fails the GitHub Actions job and stops the merge.

Why this beats a checklist. “Images must not run as root” is now versioned, code-reviewed, unit-tested and enforced identically on every build. You can prove to an auditor exactly which rule ran on which commit.

The policy input produced by Gate 6 looks like this:

```
{
  "image": {
    "user": "65532",
    "digest": "sha256:abc123...",
    "signed": true,
    "sbom_attested": true,
    "base_image": "gcr.io/distroless/static-debian12"
  }
}
```

5.2 The rules

policy/container.rego enforces five things. The first is subtle and important:

```

package main

import rego.v1

approved_base_images := {
    "gcr.io/distroless/static-debian12",
    "gcr.io/distroless/base-debian12",
}

root_users := {"", "0", "root", "0:0"}

# Fields that MUST be present. Absent fields are fail-closed: in Rego,
# `undefined != true` is itself undefined, so a missing key would silently
# skip its deny rule and the image would pass.
required_fields := {"user", "digest", "signed", "base_image"}

# RULE 0: required fields must exist (defense in depth).
deny contains msg if {
    some f in required_fields
    not f in object.keys(input.image)
    msg := sprintf("POLICY VIOLATION: required image field %q is missing from policy input", [f])
}

# RULE 1: must NOT run as root.
deny contains msg if {
    user := input.image.user
    root_users[user]
    msg := sprintf("SECURITY VIOLATION: image runs as root (user=%q)", [user])
}

# RULE 2: base image must be on the approved allowlist.
deny contains msg if {
    not approved_base_images[input.image.base_image]
    msg := sprintf("POLICY VIOLATION: base image %q is not approved", [input.image.base_image])
}

# RULE 3: must be digest-pinned.
deny contains msg if {
    not startswith(input.image.digest, "sha256:")
    msg := sprintf("POLICY VIOLATION: image must be pinned by sha256 digest, got %q", [input.image.digest])
}

# RULE 4: must be signed.
deny contains msg if {
    input.image.signed != true
    msg := "SUPPLY-CHAIN VIOLATION: image is not signed with Cosign"
}

```

policy/attestation.rego adds the SBOM requirement, also with a presence check.

The Rego gotcha worth understanding. Reading a key that does not exist yields *undefined*, and `undefined != true` is itself undefined — so the rule never fires. An early version of this policy would therefore have **allowed** an image whose `signed` field was missing entirely, rather than `false`. Asserting presence first makes the policy fail closed. This is exactly the kind of detail interviewers enjoy hearing about.

5.3 Run the policy tests

```
# Both
opa test policy/ -v
```

Expect **11 tests, all passing** — seven core rule tests and four field-presence tests. Then check the fixtures:

```
# Both
conftest test policy/tests/fixtures/compliant.json --policy policy/ --all-namespaces
conftest test policy/tests/fixtures/noncompliant.json --policy policy/ --all-namespaces
```

The second **should** report five violations. That is the correct result.

5.4 Interview questions — Phase 5

Q1. Explain the deny pattern in Rego and how Conftest turns it into a build gate. In Rego we define a partial set rule named `deny` that accumulates a message for each policy violation the input triggers. Each rule body describes a *bad* condition; when true, its message joins the set. Conftest evaluates the input and treats a non-empty `deny` as failure — it prints each message and exits non-zero. That non-zero exit fails the GitHub Actions job, which blocks the merge. An empty `deny` means compliant; any entry stops the build with a human-readable reason.

Q2. Why unit-test your policies, and what would you test? Policies are code and can have bugs — a typo could make a critical rule silently never fire, so an unsigned image passes. Unit tests assert behaviour: compliant input yields zero denials, and each malicious input trips the correct rule with the expected message. That gives regression safety when refactoring and is auditable evidence the controls work. I would test each rule’s positive and negative case, plus the fail-closed case where a field is missing entirely.

Q3. This policy checks a boolean `signed: true` that CI computed. Is that weaker than verifying the signature inside OPA? The design accounts for that: the *cryptographic* verification happens in Gate 6 with `cosign verify` bound to a specific certificate identity and OIDC issuer before the input is built — if that fails, the job stops before OPA even runs. OPA then enforces the broader compliance posture: non-root, approved base, digest pinning, evidence present. In a more advanced setup you would feed the signed attestation payload into OPA and verify the DSSE envelope in-policy, or use a Kubernetes admission controller such as Sigstore policy-controller at deploy time. Splitting responsibilities keeps each tool doing what it is best at.

Phase 6 — Vulnerability management and dashboard

Owner: Dev C.

6.1 The concept: SARIF and centralised findings

Findings that live only in CI logs get ignored. This phase makes them visible, tracked and triageable.

SARIF (Static Analysis Results Interchange Format) is a standard JSON schema for security findings. Every scanner used here — Semgrep, Hadolint, Trivy — can emit it. GitHub natively ingests SARIF: findings appear in the repository’s **Security → Code scanning alerts** tab, deduplicated, with severity, file and line, remediation advice, and a lifecycle (open → fixed → dismissed). Pull requests get inline annotations.

Why this matters. Instead of four different tool logs, triage happens in one place, tied to code, with history. A reviewer sees “this pull request introduces two new HIGH findings” right on the pull request. That is the single pane of glass this project promises.

The pipeline already uploads all three SARIF streams with distinct category: values, so GitHub keeps them as separate, non-colliding sources. This phase adds two things: a scheduled summary job that reports accumulated posture, and a portable parser you can point elsewhere later.

6.2 Enable Code Scanning

In the repository: **Settings → Code security and analysis → Code scanning → Set up → Default.**

Without this, the SARIF uploads in the pipeline have nowhere to land and those steps will warn.

6.3 The summary workflow

.github/workflows/security-summary.yaml runs every Monday and can be triggered manually:

```
on:
  schedule:
    - cron: "0 8 * * 1" # Mondays 08:00 UTC
  workflow_dispatch: {}

permissions:
  contents: read
  security-events: read
```

It runs scripts/parse_scan_results.py, which reads open alerts from the GitHub REST API, groups them by tool and severity, writes a Markdown table to the job summary, and exits non-zero if any CRITICAL alert is open.

6.4 Test it locally

```
# macOS
export GH_TOKEN=$(gh auth token)
python3 scripts/parse_scan_results.py --repo <ORG>/zt-devsecops-pipeline

# Windows (PowerShell)
$env:GH_TOKEN = gh auth token
python scripts/parse_scan_results.py --repo <ORG>/zt-devsecops-pipeline
```

Add --fail-on-critical to enforce the budget gate.

View alerts straight from the terminal:

```
# Both
gh api /repos/<ORG>/zt-devsecops-pipeline/code-scanning/alerts \
  --jq '.[ ] | {tool: .tool.name, rule: .rule.id, sev: .rule.security_severity_level}'
```

6.5 Trigger the workflow manually

```
# Both
gh workflow run security-summary.yaml
gh run watch
```

6.6 Optional — DefectDojo

scripts/upload_defectdojo.py is included but not wired into the pipeline. Use it if you later need cross-repository aggregation, SLA tracking, risk-acceptance workflows or compliance reporting. The script is deliberately portable so migrating is a change of upload target, not a pipeline rewrite.

6.7 Interview questions — Phase 6

Q1. What is SARIF and why standardise on it for a multi-tool pipeline? SARIF is a standardised JSON schema for static-analysis results. Standardising means every scanner speaks the same output format, so a single consumer — the GitHub Security tab, or a dashboard — can ingest all of them without custom parsers per tool. GitHub deduplicates findings, tracks their lifecycle and annotates pull requests inline. It turns several disconnected log streams into one queryable, historical view of security posture tied to code.

Q2. The pipeline already fails on CRITICAL/HIGH in Trivy. Why also have a separate summary job? The Trivy gate is per-build and blocks *new* code from merging, but it gives no visibility into accumulated posture — alerts that were dismissed, findings from other tools, or drift over time as new CVEs are disclosed against already-merged images. The scheduled job reads aggregated alert state on a cadence, reports trends, and can enforce an organisation-wide budget such as zero open criticals. It is the difference between a merge gate and ongoing vulnerability management.

Q3. When would you move from the GitHub Security tab to a dedicated tool like DefectDojo? The native tab is ideal for a single repository — zero infrastructure, tight pull-request integration. You outgrow it when you need cross-repository aggregation, deduplication across many products, SLA tracking and metrics, risk-acceptance workflows, ticketing integrations, or reporting for compliance frameworks. DefectDojo centralises findings from dozens of pipelines into one program-level view with triage workflows.

Phase 7 — Security audit and bypass testing

Owner: Dev D, reviewed by Dev C.

7.1 The concept: adversarial validation

A security control you have never seen *block* something is a control you do not actually trust. This phase runs three deliberate attacks and proves the pipeline stops each one.

These are **negative tests**: success means the pipeline turns **red**.

Each test targets specific gates: a vulnerable dependency should be caught by **Gate 3 (Trivy)**; a root-user container by **Gate 2 (Hadolint)** and **Gate 6 (OPA)**; an unsigned image by **Gate 6 (Cosign verify)** and deploy-time verification.

Run each on a throwaway branch. **Windows users: run these in Git Bash, not PowerShell.**

```
# Both (Windows: Git Bash)
chmod +x tests/audit/*.sh
```

7.2 Test 1 — a vulnerable dependency

```
# Both (Windows: Git Bash)
./tests/audit/01_vulnerable_dep.sh
gh run watch
```

What it does. It writes a blank import into package main so a known-vulnerable version of golang.org/x/net is actually compiled into the binary, then pushes.

Why the blank import matters. Running `go get` on an unused module does nothing — `go mod tidy` prunes it, the package never reaches the image, and Trivy would find nothing. The test would then pass while proving absolutely nothing. Forcing the import is what makes this a real test.

Expected result. Trivy flags CVE-2023-45288 (HIGH) at Gate 3; the pipeline fails.

Cleanup:

```
# Both
rm app/zz_audit_vuln.go
git checkout main
```

7.3 Test 2 — a container running as root

```
# Both (Windows: Git Bash)
./tests/audit/02_root_container.sh
```

What it does. Builds an image from `debian:12` with no `USER` directive, so it runs as root, then feeds its properties to the policy engine.

Expected result. Hadolint flags the missing `USER` at Gate 2, and OPA returns several denials at Gate 6 — running as root, unapproved base image, not digest-pinned, unsigned, and no SBOM. The pipeline fails.

Cleanup:

```
# Both
rm build/Dockerfile.insecure
git checkout main
```

7.4 Test 3 — an unsigned image

```
# macOS
export ECR_REPOSITORY=$(cd infra && terraform output -raw ecr_repository_url)
./tests/audit/03_unsigned_image.sh
```

```
# Windows – get the value in PowerShell, then run the script in Git Bash
cd infra; terraform output -raw ecr_repository_url; cd ..
# In Git Bash:
#   export ECR_REPOSITORY=<paste the value>
#   ./tests/audit/03_unsigned_image.sh
```

Expected result. `cosign verify` reports “no matching signatures” and exits non-zero.

Cleanup:

```
# Both
aws ecr batch-delete-image --repository-name zt-devsecops/go-microservice \
  --image-ids imageTag=unsigned-audit
```

7.5 Deploy-time verification

Even a signed image is verified again before it runs — trust is checked at the point of use, not just the point of creation:

```
# macOS
GITHUB_REPOSITORY=<ORG>/zt-devsecops-pipeline \
  ./security/cosign/verify.sh <ECR_URI>@sha256:<digest>
```

The script checks two things: that the signature was made by your workflow identity, and that a signed SBOM attestation exists. In Kubernetes this role is played automatically by an admission controller such as Sigstore policy-controller or Kyverno.

7.6 Evidence to capture

Take these five screenshots for the README and your portfolios:

1. The all-green pipeline run showing all six gates
2. A red run from an audit test, side by side with the green one
3. cosign verify output showing its three checks passing
4. The GitHub Security tab with findings from all three scanners
5. The ECR repository showing the image plus signature and attestation artifacts

Item 2 is the most convincing thing in the whole project. Anyone can show a green build; showing the pipeline *catching* an attack is what demonstrates the controls are real.

7.7 Interview questions — Phase 7

Q1. Why deliberately try to break your own pipeline — isn't a green build enough? A green build only proves the happy path works; it says nothing about whether the controls actually stop bad input. Negative testing proves the gates fire — that Trivy really blocks a vulnerable dependency, that OPA really rejects a root container, that Cosign really refuses an unsigned image. It catches silent misconfigurations, such as a severity threshold set wrong or a rule with a typo that never triggers, and it produces concrete evidence the posture is real. Untested controls are assumptions, not guarantees.

Q2. An image passed all six gates and was signed. Why re-verify at deploy time? Because trust must be checked at the point of use, not just the point of creation — that is the Zero-Trust principle. Between build and deploy, the reference could be swapped, or a manifest could point somewhere unexpected. Deploy-time cosign verify bound to our specific workflow identity and OIDC issuer guarantees that what is about to run is exactly the artifact our pipeline signed. In Kubernetes this is enforced automatically by an admission controller, so no unsigned or foreign image can ever schedule.

Q3. In test 3 you verify by digest. What attack does that specifically defeat? Tag-mutation and image-substitution attacks. Verifying and deploying by immutable sha256 digest means the signature is bound to exact content — an attacker who repoints a tag or pushes a look-alike image cannot produce a valid signature for our identity over their bytes. Combined with immutable tags in ECR, there is no window where the reference could point at unverified content.

Phase 8 — LinkedIn showcase

Owner: everyone. Publish only after the pipeline is green and the audits pass.

8.1 The post

The full text is in docs/linkedin-post.md. Paste it directly into LinkedIn.

Key structure: a hook aimed at DevSecOps recruiters, architecture bullets, the supply-chain core, measurable outcomes, and hashtags. The strongest line is the one about red-teaming your own pipeline — it is what separates this from a tutorial follow-along.

8.2 Posting tips

Render the Mermaid diagram to PNG and attach it as the post image; visual posts get considerably more reach. Put the repository link in the **first comment**, not the body — LinkedIn suppresses reach on posts containing external links.

All four team members should repost with a one-line personal angle (“I owned the Terraform OIDC module and learned why the sub claim is the line that matters”). That multiplies impressions and makes each person’s contribution explicit.

Best posting window: Tuesday to Thursday, 8-10am in your target recruiters' timezone.

8.3 Make the repository public

Only once the pipeline is green:

```
# Both
gh repo edit <ORG>/zt-devsecops-pipeline --visibility public --accept-visibility-change-consequences
```

Add the architecture diagram and your five screenshots to the README first. A public repository with a red badge is worse than no repository.

8.4 One thing that costs nothing and matters a lot

Have each person write three or four sentences in the README about the part they owned — what it does and why. When an interviewer opens the repository, that is the difference between “a tutorial they followed” and “a system they understood”.

Cost and cleanup

What this costs

Item	Cost
KMS customer-managed key	\$1/month (prorated hourly)
KMS key rotation	+\$1/month after the first and second rotation only
ECR storage	Free — 500 MB free tier for 12 months; this image is ~15 MB
S3 state bucket + DynamoDB lock	Fractions of a cent at this volume
GitHub Actions	Free for public repos; 2,000 min/month free on private
Sigstore, Trivy, Syft, Semgrep, OPA, Hadolint	Free and open source

Realistic total: \$1-3 per month, essentially all KMS. There is no EKS cluster, no EC2 and no load balancer in this project — those are what generate surprise bills, and none are used.

Billing trap. A KMS key pending deletion is **still billed** for the duration of its deletion window. This project sets the window to 7 days rather than the 30-day maximum, so charges stop quickly after teardown.

Teardown (Dev A)

When the screenshots are captured and the project is finished:

```
# Both
cd infra
terraform destroy          # review the plan, then confirm
```

Then remove the state backend by hand, since Terraform does not manage it:

```
# macOS
aws s3 rm "s3://$TF_BUCKET" --recursive
aws s3api delete-bucket --bucket "$TF_BUCKET"
aws dynamodb delete-table --table-name zt-devsecops-tflock
```

```
# Windows (PowerShell)
aws s3 rm "s3://$env:TF_BUCKET" --recursive
```

```
aws s3api delete-bucket --bucket $env:TF_BUCKET
aws dynamodb delete-table --table-name zt-devsecops-tflock
```

Confirm nothing is left:

```
# Both
aws ecr describe-repositories --query 'repositories[].repositoryName'
aws kms list-aliases --query "Aliases[?AliasName=='alias/zt-devsecops']"
```

Put a calendar reminder in on the day you run terraform apply. Forgotten lab resources are the most common source of unexpected cloud bills.

Troubleshooting

Symptom	Likely cause	Fix
Workflow never triggers on push	Branch is master, not main	git branch -m master main, then push
Not authorized to perform sts:AssumeRoleWithWebIdentity	Trust policy sub does not match repo/branch	Check github_org, github_repo, allowed_branches in terraform.tfvars, re-apply
Gates 3–6 skipped on a pull request	Intentional — OIDC is scoped to main	Signing runs on merge; PRs run gates 1–2 only
Docker build fails: go.sum not found	go mod tidy never committed	Dev B runs it and commits go.sum
cosign verify: none of the expected identities matched	EXPECTED_IDENTITY string mismatch	Must match org, workflow filename and ref exactly
Cannot push a tag to ECR	Tags are immutable by design	Use the commit SHA; never re-push a tag
docker: Cannot connect to the Docker daemon	Docker Desktop not running	Launch it and wait for “Engine running”
PowerShell: “running scripts is disabled”	Execution policy	Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
curl behaves oddly on Windows	It is an alias for Invoke-WebRequest	Use Invoke-RestMethod, or install real curl
./tests/audit/*.sh will not run on Windows	They are bash scripts	Run them in Git Bash, not PowerShell

Symptom	Likely cause	Fix
<code>docker inspect --format</code> prints nothing on Windows	PowerShell quote handling	Swap single quotes for double quotes
<code>opa test</code> finds no tests	Wrong working directory	Run from the repository root, not inside policy/
<code>terraform init</code> fails on the backend	TF_BUCKET not set in this shell	Re-export it, or use <code>-backend=false</code> for validation only
Trivy reports CVEs you cannot fix	Upstream has no patch yet	<code>--ignore-unfixed</code> is already set; otherwise document in .trivyignore with an owner and review date
Semgrep install fails on Windows	Python packaging issues	Run it via Docker (Phase 4.3)
SARIF upload step warns	Code Scanning not enabled	Enable it (Phase 6.2)

You are done

You have provisioned AWS with Terraform using zero stored secrets, built a hardened distroless service that runs as an unprivileged user, wired six sequential security gates into GitHub Actions, generated and signed an SBOM with no private key, enforced compliance with version-controlled policy, and then attacked your own pipeline to prove all of it works.

From here: add a Kubernetes admission controller (Sigstore policy-controller or Kyverno) so unsigned images are rejected cluster-wide at deploy time, or extend the policy set with your own organisational rules.

Do not forget `terraform destroy`.

Related documents in this repository: `docs/DAY0-SETUP.md` for the one-time organisation and repository setup, and `docs/TEAM.md` for the four-person split and day-by-day plan.